CS 473 (Spring 2025)
Homework 8 (due Apr 17 Thu 10am)

Instructions: As in previous homeworks.

Problem 8.1:

(a) (80 pts) Run the simplex method on the following linear program:

$$\begin{array}{lrcl} \text{maximize} & x_1 + x_2 & & \\ \text{s.t.} & 3x_1 + 2x_2 & \leq & 27 \\ & x_2 & \leq & 8 \\ & x_1 + 5x_2 & \leq & 35 \\ & 2x_1 + x_2 & \leq & 17 \\ & x_1, x_2 & \geq & 0. \end{array}$$

Start with the initial basic solution $(\overline{x}_1, \overline{x}_2) = (0, 0)$, and *choose x_1 as the entering variable in the first iteration.* Show the new slack form after every iteration.

(b) (20 pts) Write down the dual of the linear program from (a). What is the optimal dual solution?

Solution:

(a)

$$\begin{array}{rcrcrcr} z & = & & & x_1 & + & x_2, \\ \hline w_1 & = & 27 & - & 3x_1 & - & 2x_2, \\ w_2 & = & 8 & & & - & x_2, \\ w_3 & = & 35 & - & x_1 & - & 5x_2, \\ w_4 & = & 17 & - & 2x_1 & - & x_2, \end{array}$$

The initial solution is $(x_1, x_2, w_1, w_2, w_3, w_4) = (0, 0, 27, 8, 35, 17)$.

Iteration 1. Choose x_1, compute ratios:

$$\frac{27}{3} = 9, \quad \frac{35}{1} = 35, \quad \frac{17}{2} = 8.5,$$

Minimum is 8.5, so w_4 leaves.

$$w_4 = 17 - 2x_1 - x_2 \implies x_1 = \frac{17}{2} - \frac{1}{2}w_4 - \frac{1}{2}x_2.$$

Substitute into all other equations:

$$\begin{array}{rcrcrcr} z & = & \frac{17}{2} & - & \frac{1}{2}w_4 & + & \frac{1}{2}x_2, \\ \hline w_1 & = & \frac{3}{2} & + & \frac{3}{2}w_4 & - & \frac{1}{2}x_2, \\ w_2 & = & 8 & & & - & x_2, \\ w_3 & = & \frac{53}{2} & + & \frac{1}{2}w_4 & - & \frac{9}{2}x_2, \\ x_1 & = & \frac{17}{2} & - & \frac{1}{2}w_4 & - & \frac{1}{2}x_2. \end{array}$$

New solution is $(x_1, x_2, w_1, w_2, w_3, w_4) = (\frac{17}{2}, 0, \frac{3}{2}, 8, \frac{53}{2}, \frac{17}{2})$.

Iteration 2. Choose x_2, compute ratios:

$$\frac{3/2}{1/2} = 3, \quad \frac{8}{1} = 8, \quad \frac{53/2}{9/2} = \frac{53}{9}, \quad \frac{17/2}{1/2} = 17,$$

Minimum is 3, so w_1 leaves.

$$w_1 = \frac{3}{2} + \frac{3}{2}w_4 - \frac{1}{2}x_2 \implies x_2 = 3 + 3w_4 - 2w_1.$$

Substitute into all other equations:

$$\begin{array}{rcrcrcr} z & = & 10 & + & w_4 & - & w_1, \\ \hline x_2 & = & 3 & + & 3w_4 & - & 2w_1, \\ w_2 & = & 5 & - & 3w_4 & + & 2w_1, \\ w_3 & = & 13 & - & 13w_4 & + & 9w_1, \\ x_1 & = & 7 & - & 2w_4 & + & w_1. \end{array}$$

New solution is $(x_1, x_2, w_1, w_2, w_3, w_4) = (7, 3, 0, 5, 13, 0)$.

Iteration 3. Choose w_4, compute ratios:

$$\frac{5}{3}, \quad \frac{13}{13} = 1, \quad \frac{7}{2},$$

Minimum is 1, so w_3 leaves.

$$w_3 = 13 - 13w_4 + 9w_1 \implies w_4 = 1 - \frac{1}{13}w_3 + \frac{9}{13}w_1$$

Substitute into all other equations:

$$\begin{array}{rcrcrcl}
z & = & 11 & - & \frac{1}{13}w_3 & - & \frac{4}{13}w_1, \\
\hline
x_2 & = & 6 & + & \frac{3}{13}w_3 & + & \frac{1}{13}w_1, \\
w_2 & = & 2 & + & \frac{3}{13}w_3 & - & \frac{1}{13}w_1, \\
w_4 & = & 1 & - & \frac{1}{13}w_3 & + & \frac{9}{13}w_1, \\
x_1 & = & 5 & + & \frac{2}{13}w_4 & - & \frac{5}{13}w_1.
\end{array}$$

Thus, we obtain the optimal solution $(x_1^*, x_2^*) = (5, 6)$, $z^* = 11$.

(b) Introduce dual variables $y_1, y_2, y_3, y_4 \geq 0$ for the four constraints. The dual is

$$\begin{array}{lrl}
\min & 27y_1 + 8y_2 + 35y_3 + 17y_4, & \\
\text{s.t.} & 3y_1 + 0y_2 + y_3 + 2y_4 & \geq 1, \\
& 2y_1 + 1y_2 + 5y_3 + 1y_4 & \geq 1, \\
& y_1, y_2, y_3, y_4 & \geq 0.
\end{array}$$

From part (a), we have optimal solution $(x_1^*, x_2^*) = (5, 6)$ and $z^* = 11$. Thus, $27y_1 + 8y_2 + 35y_3 + 17y_4 \geq x_1^* + x_2^* = 10$. Using the same method in part (a), we obtain the optimal dual solution $(y_1^*, y_2^*, y_3^*, y_4^*) = (\frac{28}{91}, 0, \frac{7}{91}, 0)$.

Problem 8.2: For two points $\mathbf{p} = (p_1, \ldots, p_d) \in \mathbb{R}^d$ and $\mathbf{q} = (q_1, \ldots, q_d) \in \mathbb{R}^d$, their *rectilinear distance* is defined as

$$D(\mathbf{p}, \mathbf{q}) = |p_1 - q_1| + \cdots + |p_d - q_d|.$$

In the *rectilinear 1-center* problem in d dimensions, we are given a set P of n points in $\mathbb{R}^d$, we want to find a point $\mathbf{q} \in \mathbb{R}^d$ (not necessarily in P) that minimizes $\max_{\mathbf{p} \in P} D(\mathbf{p}, \mathbf{q})$.

(a) (30 pts) First show that the problem can be solved in $O(n)$ time for any constant dimension d. How does the running time of your algorithm grow as a function of d?

(Hint: find a small number of candidate points in P that could be the farthest point from any $\mathbf{q}$…)

(b) (70 pts) Show how to solve this problem for large (nonconstant) dimensions d by using linear programming. The number of variables and constraints should be polynomial in n and d. (Remember to justify correctness of your reduction.)

Solution:

(a) Define direction vector $s = (s_1, \ldots, s_d) \in \{+1, -1\}^d$, and projection

$$f_s(p) = \sum_{i=1}^{d} p_i s_i$$

An observation is that for any direction vector we have:

$$\sum_{i=1}^{d} |p_i - q_i| = \max_{s \in \{-1,1\}^d} \sum_{i=1}^{d} s_i(p_i - q_i)$$

because for each i one can choose s_i to be $+1$ or -1 so that $s_i(p_i - q_i) = |p_i - q_i|$. Thus,

$$D(p, q) = \max_{s \in \{-1,1\}^d} (f_s(p) - f_s(q)).$$

For each of the 2^d choices of $s \in \{+1, -1\}^d$ we scan once through all n points and find candidate points that could be the farthest from q:

$$M_s \;=\; \max_{p \in P} f_s(p), \qquad m_s \;=\; \min_{p \in P} f_s(p).$$

Once we know M_s and m_s, the best choice of $f_s(q)$ is to place

$$f_s(q) = \frac{M_s + m_s}{2}.$$

Each such scan takes $O(nd)$ time (to form each dot product $f_s(p)$ in $O(d)$ and take a max/min), so over all s this is

$$O\big(2^d \cdot n \cdot d\big).$$

Therefore, the running time grows exponentially with d. For any constant dimension d, the problem can be solved in $O(n)$ time.

(b) **Construct LP:**

For each input point $p^{(j)} = (p_1^{(j)}, \ldots, p_d^{(j)})$, and for each coordinate $i \in \{1, \ldots, d\}$, introduce a variable $t_{j,i} \geq 0$ that will represent $\big|\, q_i - p_i^{(j)} \big|$.

Introduce a single variable $r \geq 0$ to represent the maximum rectilinear distance $\max_j D(p^{(j)}, q)$.

In total, there are $O(nd)$ variables.

For each point $j \in \{1, \ldots, n\}$ and each coordinate $i \in \{1, \ldots, d\}$, enforce

$$\begin{cases} q_i - p_i^{(j)} \;\leq\; t_{j,i}, \\ p_i^{(j)} - q_i \;\leq\; t_{j,i}. \end{cases}$$

These two inequalities together force $\; t_{j,i} \geq |\, q_i - p_i^{(j)}|$.

For each point j, its rectilinear distance to q is $\sum_{i=1}^{d} |\, q_i - p_i^{(j)}|$. We enforce

$$\sum_{i=1}^{d} t_{j,i} \;\leq\; r.$$

This guarantees that r is at least as large as every $D(p^{(j)}, q)$.

That is $2nd$ constraints from the absolute-value linearization, plus n constraints from the sum $\leq r$ requirements, for a total of $O(nd)$ constraints.

The objective of this LP is to minimize r.

Proof of correctness:

($\Longrightarrow$) center with radius r:

Any choice of q and r that satisfies the LP constraints (i.e., the rectilinear 1-center) must have $\sum_i t_{j,i} \leq r$ for each j, and each $t_{j,i} \geq |q_i - p_i^{(j)}|$. Hence $\sum_i |q_i - p_i^{(j)}| \leq \sum_i t_{j,i} \leq r$, i.e. $D(p^{(j)}, q) \leq r$ for all j. Conversely, any q with $\max_j D(p^{(j)}, q) \leq r$ gives a feasible LP solution by setting $t_{j,i} = |q_i - p_i^{(j)}|$.

($\Longleftarrow$) minimal max-distance:

Since the LP minimizes r, the optimum LP value r^* is the smallest radius for which a center q exists that keeps all points within distance r^*. That is exactly the rectilinear 1-center objective.

Time complexity:

There are $O(nd)$ variables and $O(nd)$ constraints, hence we can solve the LP problem in $\text{poly}(n, d)$ time.

Problem 8.3: We are given n tasks to perform. Task i requires p_i units of power consumption for a duration of h_i hours. At any moment in time, we can perform at most 3 different tasks, and at any moment in time, the total power consumption must be at most P. A task may be preempted (possibly multiple times) at no extra cost. The problem is to devise a schedule to perform all n tasks with the minimum total number of hours.

For example: for $n = 5$ with $p_1 = 10$, $h_1 = 8.5$, $p_2 = 20$, $h_2 = 9$, $p_3 = 60$, $h_3 = 4$, $p_4 = 80$, $h_4 = 3.5$, $p_5 = 90$, $h_5 = 2$, and $P = 100$, one feasible solution is to do tasks 1 and 5 for 2 hours, then tasks 2 and 4 for 3.5 hours, then tasks 1, 2, and 3 for 4 hours, then tasks 1 and 2 for 1.5 hours, and finally task 1 for 1 hour; the total number of hours is 12. (I did not check if this is optimal. Also, for this small example, the constraint that we can do at most 3 tasks at any time is not important; but it could make a difference on larger instances.)

Describe how to solve this problem using linear programming. The number of variables and constraints should be polynomial in n.

Solution:

Let

$$\mathcal{S} \;=\; \Big\{\, S \subseteq \{1,\dots,n\} : 1 \le |S| \le 3,\; \sum_{i\in S} p_i \le P \Big\}$$

be the set of all possible combinations of tasks of size at most 3 whose total power draw is at most P. $|\mathcal{S}| = O(n^3)$.

For each combination $S \in \mathcal{S}$, introduce a variable x_S representing the total number of running hours of S.

Task i must be run for a total of h_i hours (can be split across many states). Whenever we are in state S and $i \in S$, we accrue work toward task i. Hence we enforce

$$\sum_{\substack{S\in\mathcal{S}\\ i\in S}} x_S = h_i, \quad i = 1,\dots,n.$$

$$x_S \;\ge\; 0, \quad \forall S \in \mathcal{S}.$$

Minimize the total time used:

$$\min \sum_{S\in\mathcal{S}} x_S.$$

Putting it together, we obtain the LP problem:

$$\begin{aligned} \text{Minimize} \quad & \sum_{S\in\mathcal{S}} x_S \\ \text{s.t.} \quad & \sum_{\substack{S\in\mathcal{S}\\ i\in S}} x_S \;=\; h_i, \quad i = 1,\dots,n, \\ & x_S \;\ge\; 0, \qquad\qquad \forall\, S \in \mathcal{S}. \end{aligned}$$

Algorithm:

1. Construct and solve the LP to get optimal values $\{x_S^*\}_{S\in\mathcal{S}}$.
2. List all states S with $x_S^* > 0$.
3. Build the schedule by concatenating, in any order, one time block of length x_S^* during which we run the tasks in S. Because tasks are fully preemptible, we can schedule these blocks arbitrarily.

The resulting schedule has total duration $\sum_S x_S^*$, meets the "≤ 3 tasks" and "$\le P$ power" limits by the construction above, and gives each task i exactly $\sum_{S\ni i} x_S^* = h_i$ hours of runtime. There are $|\mathcal{S}| = O(n^3)$ variables and $O(n)$ constraints, both are polynomial in n.